Politecnico di Milano

Ingegneria del Software – a.a. 2005/06

Appello del 10 luglio 2006

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi □, Ghezzi □, Morzenti □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Totale .....

Esercizio 1 (punti 9)

Il metodo statico *trasposta* ha come argomenti due interi n,m>=1, e due matrici A e T (di interi) costituite, rispettivamente, da n righe e m colonne, e da m righe e n colonne. Il metodo, senza alterare il contenuto di A, memorizza in T la matrice trasposta di A. Si rammenta che una matrice T è la trasposta di una matrice A se, e solo se, T è ottenuta da A scambiando le righe con le colonne. Si ricorda che in Java le matrici sono dichiarate come array di array. Ad esempio, A[0] è l'array che contiene la prima riga della matrice, A[1] la seconda, ecc. A[0][1] è quindi l'elemento A_{12} della matrice A.

a- Si scriva la specifica formale in JML del metodo.

//@ requires n>=1 && m>=1 && A.length == n && T.length == m &&
(\forall int i; 0<= i && i<n; A[i].length == m) && (\forall int i; 0<= i && i<m; T[i].length == n);

//@ensures (\forall int i; 0<= i && i<n;
(\forall int j; 0<= j && j<m; A[i][j]==T[j][i] && A[i][j] == \old(A[i][j])));

Opzionalmente, si poteva aggiungere anche la clausola:
\\@assignable T[*][*];

public static void trasposta (int n, int m, int [][] A, int [][] T)

b- Si trasformi la specifica JML del punto precedente sostituendo una (o più) a scelta delle precondizioni nel lancio di opportune eccezioni. Si mostri solo la parte di specifica modificata rispetto a quella definita al punto a.

Si toglie ad esempio la condizione n>=1 && m>=1 dalla requires, aggiungendola in && alla ensures. Si aggiunge inoltre la clausola signals per lanciare ad esempio una EmptyException.

b- Si scelga opportunamente un insieme minimale di casi di test funzionali, che si possa considera sufficiente per verificare il metodo statico, in base al criterio dei casi limite.
Si possono considerare i seguenti casi:
1- n ==1, m ==1
2- n>1, m ==1
3- n ==1, m>1
4- n ==m>1
5- n>1, m>1, n !=m

Esercizio 2 (punti 16)

Si consideri la classe **Agenda** che rappresenta un insieme di appuntamenti (detti anche eventi). Ogni evento è un oggetto della classe Evento, specificata informalmente al punto (a). La classe Agenda ha un metodo di inserimento, che aggiunge un evento se questo non esiste. Se l'evento è già presente in A-genda, viene sollevata l'eccezione DuplicateException. Si ha un *conflitto* quando si inserisce un evento sovrapposto anche solo per un minuto con un altro evento in Agenda. In tal caso, l'evento è comunque inserito in Agenda, ma viene segnalata l'eccezione ConflictException.

La classe ha la seguente specifica:

```
public class Agenda {
        //OVERVIEW: Un'Agenda è un insieme mutabile, anche vuoto e senza duplicazioni, di
        // Eventi.
        //@ ensures  (* costruisce un'Agenda vuota*);
        public Agenda()

        //@ requires e!=null;
        //@ ensures inAgenda(e) && (\forall Evento e1; \old(inAgenda(e1)); inAgenda(e1)) &&
        //@      (\forall Evento e1; inAgenda(e1); \old(inAgenda(e1)) || e1==e);
        //@ signals (ConflictException e) inAgenda(e) &&
        //@      (\exists Evento e1; \old(inAgenda(e1)); e.conflitto(e1)));
        //@ signals (DuplicateException e) \old(inAgenda(e)) ;
        public void inserisci(Evento e)  throws ConflictException, DuplicateException

        //@ requires e!=null;
        //@ ensures !inAgenda(e) && \old(inAgenda(e));
        //@ signals (EventDoesNotExist exc) \old(!inAgenda(e));
        public void rimuovi(Evento e) throws EventDoesNotExistException

        //@ requires e!=null;
        //@ ensures (\result == \exists Evento e1; this.eventiInData(e.dataEvento()); e1.equals(e));
        public /*@ pure @*/ boolean inAgenda(Evento e)

        //@ requires data!=null;
        //@ ensures (* \result è la lista di tutti e soli gli eventi che accadono in data d *);
        public /*@ pure @*/ ArrayList<Evento> eventiInData(Date d)
}
```

a- Completare in JML, nelle parti con i puntini, la seguente specifica della classe Evento:

```
public /*@ pure @*/ class Evento {
        //OVERVIEW: Un Evento è un oggetto immutabile, con alcune informazioni, una data e
        //un orario di inizio e di fine. L'orario è nel formato hhmm: ad es. 1630 corrisponde alle
        //ore 16.30. Due eventi sono in conflitto quando sono sovrapposti anche solo per un minuto.

        //@ requires nome!=null && data !null && oraInizio<=oraFine && oraInizio>= 0
        && oraInizio<=2359 && oraFine>= 0 && oraFine<=2359 && 0<=oraInizio%100<60 &&
         0<=oraFine%100<60;
        //@ ensures this.nomeEvento() = nome && this.dataEvento().equals(data) &&
        this.oraInizio() = oraInizio && this.oraFine() = oraFine;

        public Evento(String nome, String note, Date data, int oraInizio, int oraFine)

        //@ requires e!=null;
        //@ ensures \result ==
        //@ (e.orarioInizio()>=this.orarioInizio() && e.orarioInizio()<=this.orarioFine()) ||
        //@ (this.orarioInizio()>=e.orarioInizio() && this.orarioInizio()<=e.orarioFine())
```

```
        public boolean conflitto(Evento e)

        //@ ensures (* \result è l'orario di inizio dell'evento *);
        public int orarioInizio()

        //@ ensures (* \result è l'orario di fine dell'evento *);
        public int orarioFine()

        //@ ensures (* \result è la data dell'evento *);
        public Date dataEvento()
}
```

b- Si considerino le seguenti due estensioni (b1) e (b2) di Agenda e per ciascuna si scriva, *motivando la risposta nel modo più preciso possibile*, se verificano il principio di sostituzione:

b1- La classe AgendaSenzaConflitti estende Agenda, modificando il metodo *inserisci(Evento e)* in modo che nel caso di conflitto di *e* con altri eventi dell'Agenda, il metodo non inserisca l'evento *e*, pur lanciando l'eccezione ConflictException.

No, perchè si violerebbe la regola delle postcondizioni. Infatti, la postcondizione di *inserisci* prevede l'inserimento dell'elemento anche in caso di conflitto: per non inserire l'elemento dovremmo indebolire la postcondizione.

b2) La classe AgendaConDuplicati estende la classe Agenda aggiungendo:

- il metodo osservatore *molteplicità*, allo scopo di verificare quante volte un evento è presente in Agenda.

```
        //@ requires e!=null;
        //@ ensures (\num_of Evento e1; this.eventiInData(e.dataEvento()).contains(e1); e1.equals(e));
        public /*@ pure @*/ int molteplicità(Evento e)
```

- il nuovo metodo *inserisciMultipli*, così specificato:

```
        //@ requires e!=null;
        //@ ensures molteplicità(e) == 1+ \old(molteplicità(e)) &&
        //@ (\forall Evento e1; !e.equals(e1); molteplicità(e1)==\old(molteplicità(e1)));
        //@ signals (ConflictException exc) inAgenda(e) &&
        //@ (\forall Evento e1; \old(inAgenda(e1); inAgenda(e1)) &&
        //@      (\exists Evento e1; inAgenda(e1);  e.conflitto(e1)));
        public void inserisciMultipli (Evento e)  throws ConflictException
```

Il metodo *rimuovi* è ridefinito, in modo da aggiornare la corretta molteplicità degli elementi:

```
        //@ also
        //@ ensures (\old(inAgenda(e)) ==> molteplicità(e)==\old(molteplicità(e))-1) &&
        //@ (\forall Evento e1; inAgenda(e1) && !e.equals(e1);
        //@      molteplicità(e1)==\old(molteplicità(e1)));
        public void rimuovi(Evento e) throws EventDoesNotExistException
```

Per lo stesso motivo è ridefinito anche il metodo *inserisci,* in modo che la molteplicità di un nuovo evento inserito sia uguale a uno:

```
        //@ also
        //@ ensures \old(!inAgenda(e)) && inAgenda(e) && molteplicità(e)== 1) &&
        //@ (\forall Evento e1; inAgenda(e1) &&!e.equals(e1);
        //@      molteplicità(e1)==\old(molteplicità(e1)));
        public void inserisci(Evento e)  throws ConflictException, DuplicateException
```

NO. La ridefinizione di inserisci e rimuovi, in base alla definizione di JML, rispetta la regola dei metodi. Tuttavia, l'aggiunta del metodo inserisciMultipli comporta la violazione della regola delle proprietà, in quanto l'overview di Agenda prevede che non vi siano eventi duplicati.

c- Un possibile rep della classe Agenda è costituitocosituito da un array di eventi riempito nella parte iniziale e da un intero che indica il numero di eventi effettivamente memorizzati nell'array::

private Evento [] evList;
private int numEv;

Per ragioni di efficienza implementativa, nell'array gli eventi con la stessa data sono disposti in ordine crescente rispetto all'orario di inizio (così come definita dal metodo **orarioInizio()**), anche se non necessariamente in posizioni consecutive.
Scrivere in JML l'invariante di rappresentazione. corrispondente a questa soluzione.

```
//@ private invariant evList !=null &&
        (\forall int i; 0<=i && i< numEv; (\forall int j; i<j && j<= numEv;
                evList[i].dataEvento() == evList[j].dataEvento() ==>
                evList[i].orarioInizio() <=evList[j].orarioInizio()));
```

d- Un possibile invariante per la classe Agenda è il seguente:

Non ci sono eventi duplicati.

d1- Di che tipo di invariante si tratta? **Si tratta tipicamente di un invariante astratto (un public invariant di JML), in quanto è parte della Overview della classe.**
d2- Specificarlo formalmente in JML.

```
//@ public invariant
        (\forall Date d;;
                (\forall int i; 0<=i && i< eventiInData(d).size()-1;
                        (\forall int j; i<j && j< eventiInData(d).size();
                        ! eventiInData(d).get(i).equals(eventiInData(d).get(j)));
```

Si noti che qualora l'invariante fosse stato definito come private, non sarebbe in alcun modo possibile garantire che sia verificato solo a partire dalla specifica di Agenda (si potrebbe decidere che per ragioni di efficienza si consente di duplicare un evento in evList anche se poi questa duplicazione e' invisibile agli utilizzatori…)

d3- Stabilire, in base alla specifica della classe Agenda, se l'invariante è sempre verificato. Utilizzare eventualmente un ragionamento informale ma chiaro e conciso.

Intuitivamente, l'invariante dovrebbe essere verificato, in quanto:

1) **il costruttore inizializza un'Agenda senza eventi (e quindi senza duplicati);**
2) **se il metodo inserisci viene chiamato su un'Agenda senza duplicati, la specifica del metodo sembra tale per cui un evento è inserito solo se non è equals a uno esistente. Pertanto, l'Agenda risultante continua a non avere duplicati.**
3) **Nessun altro metodo oltre al costruttore e a inserisci è in grado di aggiungere eventi a un'Agenda.**

Si osservi tuttavia che, formalmente, la specifica di inserisci, nel caso di DuplicateException non garantisce nulla… Pertanto, il punto (2) formalmente non può essere garantito e quindi l'invariante non è verificabile in modo formale.